

Manual del Programador

Sistema IoT de Monitorización y Control de Agua

Manual del Programador - Sistema IoT de Monitorización y Control de Agua

1. Arquitectura General y Topología de Red

El Sistema IoT de Monitorización y Control de Agua se basa en un modelo de arquitectura distribuida desacoplada, utilizando el protocolo de telemetría MQTT sobre TCP/IP como núcleo de mensajería asíncrona. La topología de red integra un microcontrolador ESP32 de borde, una plataforma de automatización intermedia en Node-RED y una aplicación de cliente web que actúa como panel de supervisión (Frontend).

La orquestación lógica distribuye las responsabilidades operativas en cuatro niveles claros:

1. **Nivel de Instrumentación:** El microcontrolador ESP32 interactúa directamente con los transductores de hardware mediante interrupciones físicas de baja latencia para realizar el conteo de volumen y modular los estados lógicos de potencia.
2. **Nivel de Mensajería (Broker MQTT):** Utiliza el servidor público de HiveMQ para coordinar el enrutamiento y la distribución de publicaciones bajo un modelo de suscripción síncrona.
3. **Nivel de Backend (Node-RED):** Realiza las integraciones numéricas temporales necesarias para determinar los consumos volumétricos acumulados y gestionar los ciclos cronológicos de reinicio del sistema.
4. **Nivel de Presentación (Web UI):** Consume los datos serializados en JSON mediante una conexión persistente por WebSockets seguros (WSS), modificando el DOM de forma asíncrona para reflejar las telemetrías.

2. Especificaciones de Hardware y Asignación de Pines

Para asegurar la estabilidad del microcontrolador ante las perturbaciones de conmutación electromagnética de cargas inductivas, las conexiones físicas se configuran bajo la siguiente distribución:

Componente Relacionado	Pin Físico ESP32	Nivel Lógico	Función Técnica
Módulo Relé (VCC)	VIN / 3.3V	5.0V DC	Alimentación en el primer pin del módulo ESP-32
Módulo Relé (GND)	GND	0.0V Sólido	Referencia de masa del sistema
Módulo Relé (IN1)	GPIO 25	3.3V TTL	Control digital de Electroválvula
Sensor Caudal YF-S201 (VCC)	VIN / 5V	5.0V DC	Alimentación del circuito de efecto Hall
Sensor Caudal YF-S201 (GND)	GND	0.0V Sólido	Referencia de masa común
Sensor Caudal YF-S201 (Signal)	GPIO 27	3.3V TTL	Entrada de pulsos por interrupción

Condición del Circuito Eléctrico: Es mandatorio acoplar una resistencia de pull-up externa con un valor nominal de 10kΩ entre el pin de señal del sensor de caudal y el riel de alimentación regulado de 3.3V de la ESP32.

3. Arquitectura y Análisis de Firmware (ESP32)

El script desarrollado con la extensión .INO implementa un esquema asíncrono no bloqueante basado en la función nativa ``millis()`` para evitar retardos en la pila de red.

3.1 Rutinas de Interrupción por Hardware (ISR)

Para mitigar la pérdida de datos provocada por los tiempos de latencia en la transmisión

WiFi, el sensor es analizado en segundo plano por hardware a través de una función asíncrona direccionada a la memoria RAM interna (`IRAM\_ATTR`):

```
void IRAM_ATTR pulseCounter() {  
    pulseCount++;  
}
```

3.2 Procesamiento de Muestreo Métrico

A intervalos fijos de 1000 milisegundos, el bucle principal detiene temporalmente las interrupciones de forma segura para copiar la variable del acumulador de pulsos, reiniciarla y calcular el caudal neto aplicando el coeficiente de conversión específico para el sensor YF-S201 (450 pulsos por litro circulante):

```
noInterrupts();  
unsigned long pulses = pulseCount;  
pulseCount = 0;  
interrupts();  
  
float flow_l_min = (pulses * 60.0) / PULSES_PER_LITER;
```

3.3 Estructura de Mensajes JSON y Publicación

Las variables locales de telemetría son codificadas en formato JSON utilizando memoria estática fija para evitar fallos por fragmentación de heap en ejecuciones prolongadas:

```
StaticJsonDocument<200> doc;  
doc["flow_l_min"] = flow_l_min;  
doc["pulses"] = pulses;  
doc["timestamp"] = now;  
  
char buf[128];  
size_t n = serializeJson(doc, buf);  
client.publish("casa/agua/caudal", buf, n);
```

3.4 Reglas Lógicas de Seguridad Embebidas

El firmware ejecuta dos condicionales de control locales independientes para garantizar la

integridad física ante pérdidas de conexión externas:

- **Fuga en Válvula Cerrada:** Si `flow_I_min > 0.1` y la variable `valveState` es igual a falso (`false`), se publica el identificador "Fuga_detectada_valvula_cerrada" en el tópico de alerta.
- **Fuga por Tiempo Excesivo:** Si `flow_I_min > 0.1` de forma persistente, se evalúa la marca de tiempo inicial. Al superar los 60,000 milisegundos (`LEAK_TIME_THRESHOLD`), el firmware commuta directamente el pin físico del relé (`digitalWrite(RELAY_VALVE_PIN, LOW)`), actualiza las banderas lógicas internas a apagado y despacha el mensaje "Fuga_Tiempo_Excesivo".

4. Lógica de Negocio y Procesamiento Intermedio (Node-RED)

El motor lógico intermedio suscribe su flujo a los tópicos MQTT crudos para consolidar la información histórica del sistema.

4.1 Algoritmo de Integración Volumétrica

El nodo JavaScript de tipo función realiza el cálculo matemático integral del volumen inyectado evaluando la diferencia en milisegundos (`deltaMs`) entre los paquetes entrantes:

```
let diario = flow.get('consumoDiario') || 0;
let semanal = flow.get('consumoSemanal') || 0;
let ultimoTimestamp = flow.get('ultimoTimestamp') || Date.now();

let ahora = Date.now();
let deltaMs = ahora - ultimoTimestamp;
flow.set('ultimoTimestamp', ahora);

if (deltaMs > 0 && deltaMs < 5000) {
  let flow_I_min = msg.payload.flow_I_min || 0;
  let litros = flow_I_min * (deltaMs / 60000.0);
  diario += litros;
  semanal += litros;
  flow.set('consumoDiario', diario);
  flow.set('consumoSemanal', semanal);
}
```

```
}
```

4.2 Manejo del Ciclo de Vida de los Acumuladores

El script evalúa de forma asíncrona la fecha provista por el servidor local del sistema para reiniciar a cero los objetos en el cambio de jornada o ciclo semanal (Lunes):

```
let hoy = new Date();
let ultimoDia = flow.get('ultimoDia') || hoy.getDate();
if (hoy.getDate() !== ultimoDia) {
  diario = 0;
  flow.set('consumoDiario', diario);
  flow.set('ultimoDia', hoy.getDate());
  if (hoy.getDay() === 1) {
    semanal = 0;
    flow.set('consumoSemanal', semanal);
  }
}
```

5. Arquitectura del Frontend Web y Comunicación Async

La interfaz gráfica del usuario se compone de un archivo HTML estático acompañado de un script lógico estructurado en JavaScript asíncrono puro (`app.js`). El ecosistema web se comunica con el Broker remoto a través del puerto seguro de WebSockets (8884).

5.1 Suscripción de Tópicos Operativos

Al completarse el evento `DOMContentLoaded`, el cliente web inicializa la conexión asíncrona e invoca de manera explícita la escucha de los tópicos del bus de datos:

- `casa/agua/caudal`: Retorna telemetrías crudas para refrescar el medidor circular gráfico.
- `casa/agua/estado`: Envía la confirmación del estado lógico de la válvula física para sincronizar los controles deslizantes.
- `casa/agua/alerta`: Transporta los códigos lógicos de fallas para disparar el

renderizado dinámico del banner de peligro.

- `casa/agua/consumo\_procesado`: Suministra los volúmenes integrados para los paneles laterales e históricos.

5.2 Despacho de Comandos de Control

La manipulación remota de la electroválvula se gestiona transmitiendo el comando correspondiente hacia los tópicos de control con nivel de calidad de servicio QoS 1:

```
valveSwitch.addEventListener('change', (e) => {  
  const command = e.target.checked ? 'ON' : 'OFF';  
  if (client && client.connected) {  
    client.publish('casa/agua/valvula', command, { qos: 1 });  
  }  
});
```

6. Gestión de Errores y Tolerancia a Fallos en Código

Para mitigar los efectos de pérdidas de alimentación eléctrica o desconexiones de red, el firmware de la ESP32 implementa una rutina persistente de reconexión dentro de la función cíclica principal:

```
void reconnect() {  
  while (!client.connected()) {  
    String clientId = "ESP32-Agua-" + String((uint32_t)ESP.getEfuseMac(), HEX);  
    if (client.connect(clientId.c_str())) {  
      client.subscribe("casa/agua/valvula");  
      publishState();  
    } else {  
      delay(2000);  
    }  
  }  
}
```